

SMART CAMPUS PARKING & TRAFFIC INTELLIGENCE SYSTEM

CSA0925-PROGRAMMIN IN JAVA

Report

L. Jeyakannan

192525376

E .Lakshmi Narayanan

192524249

GitHub links

<https://github.com/Jeyakannan-473/Smart-campus-parking-system---Java>

<https://github.com/Lakshminarayanan2211/LakshmiNarayanancsa0925>

1.PSEUDO CODE

1.1 Application Start and Authentication

```
BEGIN
LOAD database driver
OPEN AWT login window
READ username and password
IF credentials are valid and active THEN
    OPEN dashboard
    LOAD parking, occupancy, traffic and alert summaries
ELSE
    DISPLAY login error
END IF
END
```

Algorithmic

```
Class.forName("com.mysql.cj.jdbc.Driver");

Connection con =
DriverManager.getConnection("jdbc:mysql://localhost:3306/smart_campus_parking","root","password");

String sql = "SELECT * FROM users WHERE username=? AND password=? AND status='ACTIVE'";

PreparedStatement ps = con.prepareStatement(sql);

ps.setString(1, username);

ps.setString(2, password);

ResultSet rs = ps.executeQuery();

if (rs.next()) {

    System.out.println("Login successful");

    new Dashboard();

} else {

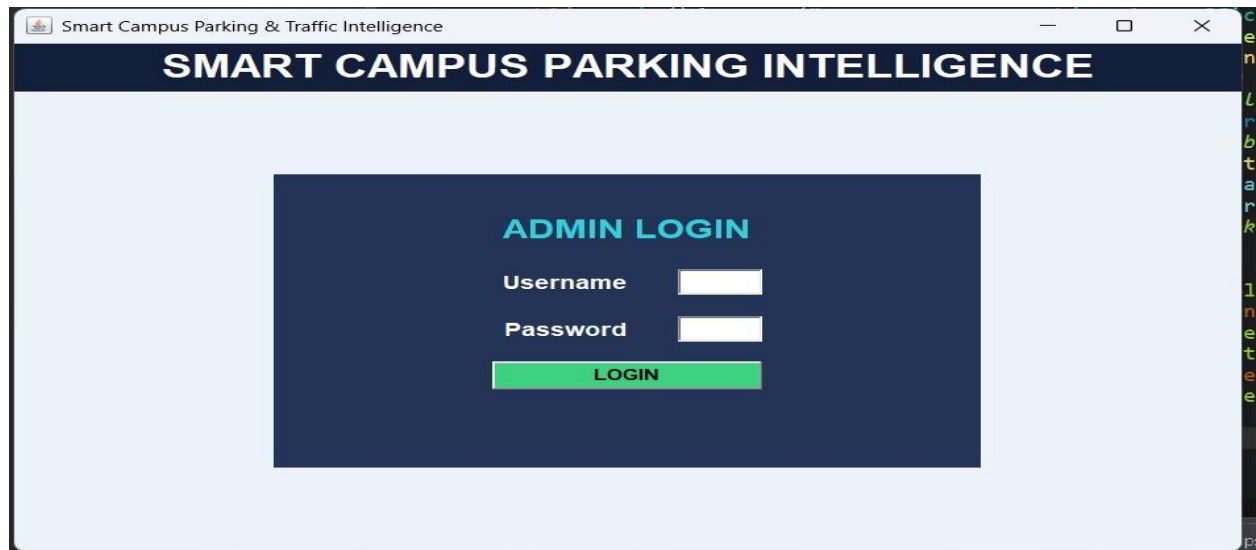
    System.out.println("Invalid username or password");}
```

```

package smartcampus;
import java.awt.*; import java.awt.event.*; import java.sql.*;
public class Dashboard extends Frame {
    CardLayout cards=new CardLayout(); Panel content=new Panel(cards); Label status=new Label("Ready");
    public Dashboard(){super("Smart Campus Parking Intelligence - AWT");setSize(1280,780);setLayout(new BorderLayout());setBackground(
    Panel side=new Panel(new GridLayout(0,1,5,5));side.setBackground(Color.NAVY);side.setPreferredSize(new Dimension(205,0));
    String[] names={"DASHBOARD","PARKING MAP","RESERVATIONS","ENTRY / EXIT","USERS","VEHICLES","PASSES","VIOLATIONS","REPORTS","SETTING
    Component[] views={new Home(this),new ParkingMap(this),new Reservation(this),new Session(this),new Users(this),new Vehicles(this),
    for(int i=0;i<names.length;i++){final int x=i;Button b=new Button(names[i],i==0?Color.CYAN:Color.WHITE);side.add(b);content.add(views[i],
    add(side,BorderLayout.WEST);add(content,BorderLayout.CENTER);
    Panel bottom=new Panel(new BorderLayout());bottom.setBackground(Color.NAVY);status.setForeground(Color.WHITE);
    bottom.add(status,BorderLayout.WEST);bottom.add(new Label("AWT • JDBC • MYSQL",12,Color.CYAN),BorderLayout.EAST);add(bottom,BorderLayout
    }
    class Home extends Panel{
        Dashboard d; Label total=new Label("0",28,Color.CYAN),free=new Label("0",28,Color.GREEN),occ=new Label("0",28,Color.ORANGE),util=new Label("0%",
    Home(Dashboard d){this.d=d;setLayout(new BorderLayout(12,12));setBackground(Color.BG);Panel head=new Panel(new BorderLayout());head.set
    Panel grid=new Panel(new GridLayout(2,3,12,12));grid.add(tile("TOTAL SLOTS",total,Color.CARD));
    grid.add(tile("AVAILABLE",free,Color.CARD));grid.add(tile("OCCUPIED",occ,Color.CARD));grid.add(tile("UTILIZATION",util,Color.CARD));grid.ad
    alerts.setEditable(false);alerts.setBackground(Color.WHITE);Panel a=new Panel(new BorderLayout());a.add(new Label("LIVE ALERTS",16,
    Panel tile(String name,Label value,Color bg){Panel p=new Panel(180,110);p.setBackground(bg);p.setLayout(new BorderLayout());p.add(
    void load(){try(Connection c=DB.getConnection()){
        {ResultSet r=c.createStatement().executeQuery("select count(*) total,sum(status='Available') free,sum(status='Occupied') occ fr
        }r=c.createStatement().executeQuery("select coalesce(round(avg(congestion_level),0),0) from traffic_records where record_time>=
        if(r.next())traffic.setText(r.getString(1)+" / 100");r=c.createStatement().executeQuery("select coalesce(round(avg(occupied_slo
        if(r.next())pred.setText(r.getString(1)+"%");r=c.createStatement().executeQuery("select message from alerts where is_read=0 ord
        }catch(Exception e){alerts.setText("Database not connected: "+e.getMessage());}}
    }
}

```

Output:



1.2 Smart Slot Recommendation

```
BEGIN
GET all parking slots
FILTER slots whose status is AVAILABLE
IF no slot exists THEN
    DISPLAY no-availability message
    STOP
END IF
FOR each available slot
    CALCULATE suitability score from current parking information
END FOR
SELECT highest-scoring slot
DISPLAY recommendation
OPTIONALLY STORE recommendation log
END
```

Algorithmic Logic Code

```
String sql = "SELECT slot_id, zone_id FROM parking_slots " +
    "WHERE status='AVAILABLE'";
```

```
Statement st = con.createStatement();
ResultSet rs = st.executeQuery(sql);
```

```
int bestSlot = -1;
int bestScore = -1;
```

```
while (rs.next()) {
    int slotId = rs.getInt("slot_id");
    int zoneId = rs.getInt("zone_id");
```

```
    int score = 100 - (zoneId * 5);
```

```
    if (score > bestScore) {
        bestScore = score;
        bestSlot = slotId;
```

```
    }
```

```
}
```

```
if (bestSlot == -1) {
    System.out.println("No parking slot available");
} else {
    System.out.println("Recommended Slot: " + bestSlot);
}
```

```

package smartcampus;
import java.awt.*;import java.sql.*;
public class Reservation extends Panel{
    TextField plate=UI.field(""),slot=UI.field(""),start=UI.field("2026-09-02 10:00:00"),end=UI.field("2026-09-02 12:00:00"); TextArea out=new TextArea();
    Reservation(Frame f){setLayout(new BorderLayout(10,10));setBackground(UI.BG);add(UI.Label("PARKING RESERVATION",22,UI.NAVY),BorderLayout.NORTH);Panel form=new Pa
    void save(){try(Connection c=DB.get()){PreparedStatement q=c.prepareStatement("select vehicle_id from vehicles where plate_number=?");
    q.setString(1,plate.getText().trim());ResultSet vr=q.executeQuery();if(!vr.next()){out.setText("Vehicle not found.");return;
    }int vid=vr.getInt(1);q=c.prepareStatement("select slot_id from parking_slots where slot_code=? and status='Available'");
    q.setString(1,slot.getText().trim());ResultSet sr=q.executeQuery();if(!sr.next()){out.setText("Slot not available.");
    return;}int sid=sr.getInt(1);q=c.prepareStatement("select reservation_id from reservations where slot_id=? and status='Confirmed' and start_time < ? and end_time
    q.setInt(1,sid);q.setString(2,end.getText());q.setString(3,start.getText());if(q.executeQuery().next()){out.setText("Conflict: slot already reserved in this time
    return;}q=c.prepareStatement("insert into reservations(vehicle_id,slot_id,start_time,end_time,status) values(?,?,?,?,'Confirmed')");q.setInt(1,vid);q.setInt(2,sid);
    c.createStatement().executeUpdate("update parking_slots set status='Reserved' where slot_id="+sid);out.setText("Reservation confirmed for "+plate.getText()+" at
    }
}

```

Output;

1.3 Reservation

BEGIN

READ user, vehicle, slot and requested time range

VALIDATE required fields

CHECK slot existence and current status

CHECK overlapping reservations

IF conflict exists **THEN**

REJECT reservation

ELSE

INSERT reservation

DISPLAY confirmation

END IF

END

Algorithmic Logic Code

String checkSql =

 "SELECT reservation_id FROM reservations " +

 "WHERE slot_id=? AND start_time < ? AND end_time > ?";

PreparedStatement check = con.prepareStatement(checkSql);

check.setInt(1, slotId);

check.setString(2, endTime);

check.setString(3, startTime);

ResultSet rs = check.executeQuery();

if (rs.next()) {

 System.out.println("Reservation conflict");

} else {

 String insertSql =

 "INSERT INTO reservations " +

 "(user_id, vehicle_id, slot_id, start_time, end_time) " +

 "VALUES (?, ?, ?, ?, ?)";

 PreparedStatement ps = con.prepareStatement(insertSql);

```

ps.setInt(1, userId);
ps.setInt(2, vehicleId);
ps.setInt(3, slotId);
ps.setString(4, startTime);
ps.setString(5, endTime);

ps.executeUpdate();

System.out.println("Reservation created successfully");
}

```

```

package smartcampus;
import java.awt.*;import java.sql.*;
public class Reservation extends Panel{
    TextField plate=UI.field(""),slot=UI.field(""),start=UI.field("2026-09-02 10:00:00"),end=UI.field("2026-09-02 12:00:00");
    TextArea out=new TextArea();
    Reservation(Frame f){setLayout(new BorderLayout(10,10));setBackground(UI.BG);add(UI.Label("PARKING RESERVATION",22,UI.NAVY),BorderLayout.NORTH);
    Panel form=new Pa
    void save(){try(Connection c=DB.get()){PreparedStatement q=c.prepareStatement("select vehicle_id from vehicles where plate_number=?");
    q.setString(1,plate.getText().trim());ResultSet vr=q.executeQuery();if(!vr.next()){out.setText("Vehicle not found.");return;
    }int vid=vr.getInt(1);q=c.prepareStatement("select slot_id from parking_slots where slot_code=? and status='Available'");
    q.setString(1,slot.getText().trim());ResultSet sr=q.executeQuery();if(!sr.next()){out.setText("Slot not available.");
    return;}int sid=sr.getInt(1);q=c.prepareStatement("select reservation_id from reservations where slot_id=? and status='Confirmed' and start_time < ? and end_time
    q.setInt(1,sid);q.setString(2,end.getText());q.setString(3,start.getText());if(q.executeQuery().next()){out.setText("Conflict: slot already reserved in this time
    return;}q=c.prepareStatement("insert into reservations(vehicle_id,slot_id,start_time,end_time,status) values(?,?,?,?,'Confirmed')");q.setInt(1,vid);q.setInt(2,si
    c.createStatement().executeUpdate("update parking_slots set status='Reserved' where slot_id="+sid);out.setText("Reservation confirmed for "+plate.getText()+" at
    }
}

```

Output:

REPORTS & ANALYTICS		
DASHBOARD		DATE AVG CONGESTION RECORDS 2026-09-02 41 3 4
PARKING MAP	ZONE OCCUPANCY	
RESERVATIONS	REVENUE	
ENTRY / EXIT		
USERS	TRAFFIC	
VEHICLES		
PASSES	VIOLATIONS	
VIOLATIONS		
REPORTS	PREDICTION	
SETTINGS		

1.4 Vehicle Entry

```
BEGIN
READ vehicle and selected slot
VALIDATE vehicle registration
CHECK slot availability
IF unavailable THEN DISPLAY error
ELSE
    CREATE parking session with entry time
    UPDATE slot to OCCUPIED
    DISPLAY entry confirmation
END IF
END
```

Algorithmic Logic Code

```
String checkSql =
    "SELECT status FROM parking_slots WHERE slot_id=?";

PreparedStatement check = con.prepareStatement(checkSql);
check.setInt(1, slotId);

ResultSet rs = check.executeQuery();

if (rs.next() && rs.getString("status").equals("AVAILABLE")) {

    String sessionSql =
        "INSERT INTO parking_sessions " +
        "(vehicle_id, slot_id, entry_time) VALUES (?, ?, NOW())";

    PreparedStatement ps = con.prepareStatement(sessionSql);
    ps.setInt(1, vehicleId);
    ps.setInt(2, slotId);
    ps.executeUpdate();

    String updateSql =
        "UPDATE parking_slots SET status='OCCUPIED' WHERE slot_id=?";

    PreparedStatement update = con.prepareStatement(updateSql);
    update.setInt(1, slotId);
    update.executeUpdate();

    System.out.println("Vehicle entry recorded");
} else {
    System.out.println("Selected slot is unavailable");
}
```



```

package smartcampus;
import java.awt.*;import java.sql.*;
public class Vehicles extends Panel{TextField plate=UI.field(""),owner=UI.field(""),
type=UI.field("Car");TextArea out=new TextArea();Vehicles(Frame f)
{setLayout(new BorderLayout(8,8));setBackground(UI.BG);add(UI.Label("VEHICLE MANAGEMENT - CRUD",22,UI.NAVY),
BorderLayout.NORTH);Panel p=new Panel(new GridLayout(0,2,5,5));p.add(new Label("Plate Number"));
p.add(plate);p.add(new Label("Owner User ID"));p.add(owner);p.add(new Label("Vehicle Type"));
p.add(type);Button add=UI.button("ADD VEHICLE",UI.GREEN),del=UI.button("DELETE PLATE",UI.RED),
load=UI.button("REFRESH",UI.CYAN);p.add(add);p.add(del);p.add(load);add(p,BorderLayout.WEST);
out.setEditable(false);add(out,BorderLayout.CENTER);add.addActionListener(e->add());
del.addActionListener(e->del());load.addActionListener(e->load());load();void add()
{try(Connection c=DB.get();PreparedStatement
p=c.prepareStatement("insert into vehicles(user_id,plate_number,vehicle_type) values(?,?,?)")
{p.setInt(1,Integer.parseInt(owner.getText()));p.setString(2,plate.getText());p.setString(3,type.getText());
p.executeUpdate();load();}catch(Exception e)
{out.setText(e.getMessage());}void del(){try(Connection c=DB.get();
PreparedStatement p=c.prepareStatement("delete from vehicles where plate_number=?")
{p.setString(1,plate.getText());p.executeUpdate();load();}catch(Exception e)
{out.setText(e.getMessage());}void load(){try(Connection c=DB.get()
{ResultSet r=c.createStatement().executeQuery("select vehicle_id,user_id,plate_number,vehicle_type from vehicles order by vehicle_id");
StringBuilder s=new StringBuilder("ID | USER ID | PLATE | TYPE\n");while(r.next())s.append(r.getInt(1)).
append(" | ").append(r.getInt(2)).append(" | ").append(r.getString(3)).append(" | ").append(r.getString(4)).append('\n');
out.setText(s.toString());}catch(Exception e){out.setText("DB error: "+e.getMessage());}}

```

Output :

VEHICLE MANAGEMENT □ CRUD

Plate Number	Tn12322	ID USER ID PLATE TYPE 1 1 TN01AA1234 Car 2 2 TN01BB5678 Bike 3 2 TN01CC9999 Car 18 3 TN453443 Car
Owner User ID	3	
Vehicle Type	Car	
ADD VEHICLE DELETE PLATE REFRESH		

1.5 Vehicle Exit and Fee

BEGIN

FIND active session for vehicle

IF no active session THEN DISPLAY error and STOP

RECORD exit time

CALCULATE duration

CALCULATE parking fee

UPDATE session


```
CREATE payment record
UPDATE slot to AVAILABLE
DISPLAY exit confirmation and fee
END
```

Algorithmic Logic Code

```
String sql =
    "SELECT session_id, slot_id, entry_time " +
    "FROM parking_sessions " +
    "WHERE vehicle_id=? AND exit_time IS NULL";

PreparedStatement ps = con.prepareStatement(sql);
ps.setInt(1, vehicleId);

ResultSet rs = ps.executeQuery();

if (rs.next()) {

    int sessionId = rs.getInt("session_id");
    int slotId = rs.getInt("slot_id");

    String feeSql =
        "SELECT TIMESTAMPDIFF(HOUR, entry_time, NOW()) " +
        "FROM parking_sessions WHERE session_id=?";

    PreparedStatement feePs = con.prepareStatement(feeSql);
    feePs.setInt(1, sessionId);

    ResultSet feeRs = feePs.executeQuery();

    if (feeRs.next()) {
        int hours = feeRs.getInt(1);

        if (hours < 1)
            hours = 1;

        double fee = hours * 20.0;

        String updateSession =
            "UPDATE parking_sessions SET exit_time=NOW(), fee=? " +
            "WHERE session_id=?";

        PreparedStatement update = con.prepareStatement(updateSession);
        update.setDouble(1, fee);
        update.setInt(2, sessionId);
        update.executeUpdate();

        String paymentSql =
            "INSERT INTO payments (session_id, amount, payment_status) " +
```

```
"VALUES (?, ?, 'PAID')");
```

```
PreparedStatement payment = con.prepareStatement(paymentSql);  
payment.setInt(1, sessionId);  
payment.setDouble(2, fee);  
payment.executeUpdate();
```

```
String slotSql =
```

```
"UPDATE parking_slots SET status='AVAILABLE' WHERE slot_id=?";
```

```
PreparedStatement slot = con.prepareStatement(slotSql);  
slot.setInt(1, slotId);  
slot.executeUpdate();
```

```
System.out.println("Exit completed");  
System.out.println("Parking Fee: " + fee);
```

```
}
```

```
} else {
```

```
System.out.println("No active parking session found");
```

```
}
```

```
package smartcampus;  
import java.awt.*;import java.sql.*;  
public class Session extends Panel{  
    TextField plate=UI.field(""), slot=UI.field(""), sessionId=UI.field("");TextArea out=new TextArea();  
    Session(Frame f){setLayout(new BorderLayout(10,10));setBackground(UI.BG);  
    add(UI.Label("VEHICLE ENTRY / EXIT + AUTOMATIC FEE",22,UI.NAVY),BorderLayout.NORTH);Panel p=new Panel(new GridLayout(0,2,8,8));  
    p.add(new Label("Entry Plate"));p.add(plate);p.add(new Label("Slot Code"));p.add(slot);Button in=UI.button("VEHICLE ENTRY",UI.GREEN);  
    p.add(in);p.add(new Label(""));p.add(new Label("Session ID for Exit"));p.add(sessionId);Button ex=UI.button("VEHICLE EXIT / CALCULATE FEE",UI.ORANGE);  
    p.add(ex);add(p,BorderLayout.WEST);out.setEditable(false);add(out,BorderLayout.CENTER);in.addActionListener(e->entry());ex.addActionListener(e->exit());}  
    void entry(){try(Connection c=DB.get()){PreparedStatement p=c.prepareStatement("select vehicle_id from vehicles where plate_number=?");  
    p.setString(1,plate.getText());ResultSet r=p.executeQuery();if(!r.next()){out.setText("Vehicle not found");return;}int vid=r.getInt(1);  
    p=c.prepareStatement("select slot_id from parking_slots where slot_code=? and status in ('Available','Reserved')");p.setString(1,slot.getText());  
    r=p.executeQuery();if(!r.next()){out.setText("Slot unavailable");return;}int sid=r.getInt(1);  
    p=c.prepareStatement("insert into parking_sessions(vehicle_id,slot_id,entry_time,status) values(?,?,now(),'Active')",Statement.RETURN_GENERATED_KEYS);  
    p.setInt(1,vid);p.setInt(2,sid);p.executeUpdate();ResultSet k=p.getGeneratedKeys();k.next();  
    int id=k.getInt(1);c.createStatement().executeUpdate("update parking_slots set status='Occupied' where slot_id="+sid);  
    out.setText("ENTRY SUCCESS. Session ID = "+id);sessionId.setText(""+id);}catch(Exception e){out.setText("Error: "+e.getMessage());}  
    void exit(){try(Connection c=DB.get()){int id=Integer.parseInt(sessionId.getText());  
    CallableStatement p=c.prepareCall("{call calculate_parking_fee(?,?)}");  
    p.setInt(1,id);p.registerOutParameter(2,Types.DECIMAL);p.execute();double fee=p.getDouble(2);  
    out.setText("EXIT COMPLETE\nSession: "+id+"\nParking Fee: Rs. "+String.format("%.2f",fee)+"\nPayment record created.");  
    }catch(Exception e){out.setText("Exit error: "+e.getMessage());}  
    }
```

OUTPUT:

VEHICLE ENTRY / EXIT + AUTOMATIC FEE	
Entry Plate	
Slot Code	
VEHICLE ENTRY	
Session ID for Exit	
VEHICLE EXIT / CALCULATE FEE	

1.6 Digital Pass

```
BEGIN
READ user and vehicle
VALIDATE vehicle
GENERATE unique pass code
SET issue and expiry time
INSERT pass record
DISPLAY formatted pass
END
```

Algorithmic Logic Code

```
String passCode = "PASS" + System.currentTimeMillis();
```

```
String sql =
    "INSERT INTO parking_passes " +
    "(user_id, vehicle_id, pass_code, issue_time, expiry_time) " +
    "VALUES (?, ?, ?, NOW(), DATE_ADD(NOW(), INTERVAL 24 HOUR));"
```

```
PreparedStatement ps = con.prepareStatement(sql);
```

```
ps.setInt(1, userId);
ps.setInt(2, vehicleId);
ps.setString(3, passCode);
```

```
ps.executeUpdate();
```

```
System.out.println("Digital Parking Pass Generated");
```

```
System.out.println("Pass Code: " + passCode);
```

```
package smartcampus;
import java.awt.*;import java.sql.*;import java.util.UUID;
public class Pass extends Panel{TextField plate=UI.field("");
    TextArea out=new TextArea();Pass(Frame f){setLayout(new BorderLayout(8,8));
    setBackground(UI.BG);add(UI.Label("DIGITAL PARKING PASS",22,UI.NAVY),BorderLayout.NORTH);
    Panel p=new Panel(new FlowLayout());p.add(new Label("Vehicle Plate"));p.add(plate);
    Button b=UI.button("GENERATE 24-HOUR PASS",UI.PURPLE);p.add(b);add(p,BorderLayout.WEST);
    out.setEditable(false);add(out,BorderLayout.CENTER);b.addActionListener(e->make());
    }void make(){try(Connection c=DB.get()){PreparedStatement p=c.prepareStatement("select vehicle_id from vehicles where plate_number=?");
    p.setString(1,plate.getText());ResultSet r=p.executeQuery();
    if(!r.next()){out.setText("Vehicle not found");
    return;}String code="PASS-"+UUID.randomUUID().toString().substring(0,8).toUpperCase();p=c.prepareStatement("insert into parking_pass");
    p.setInt(1,r.getInt(1));p.setString(2,code);p.executeUpdate();
    out.setText("=====\n SMART CAMPUS DIGITAL PASS\n=====\nPass Code : "+code+"\n");
    }catch(Exception e){out.setText(e.getMessage());}}
```

DIGITAL PARKING PASS

Vehicle Plate

TN01AA1234

GENERATE 24-HOUR PASS

1.7 Violation

BEGIN

READ vehicle, violation type and details

VALIDATE input

DETERMINE fine amount

INSERT violation record

DISPLAY confirmation

END

Algorithmic Logic Code

```
double fine = 0;
```

```

if (violationType.equals("NO_PARKING")) {
    fine = 500;
} else if (violationType.equals("WRONG_SLOT")) {
    fine = 300;
} else if (violationType.equals("OVERTIME")) {
    fine = 200;
}

String sql =
    "INSERT INTO violations " +
    "(vehicle_id, violation_type, details, fine_amount) " +
    "VALUES (?, ?, ?, ?)";

```

```

PreparedStatement ps = con.prepareStatement(sql);

```

```

ps.setInt(1, vehicleId);
ps.setString(2, violationType);
ps.setString(3, details);
ps.setDouble(4, fine);

```

```

ps.executeUpdate();

```

```

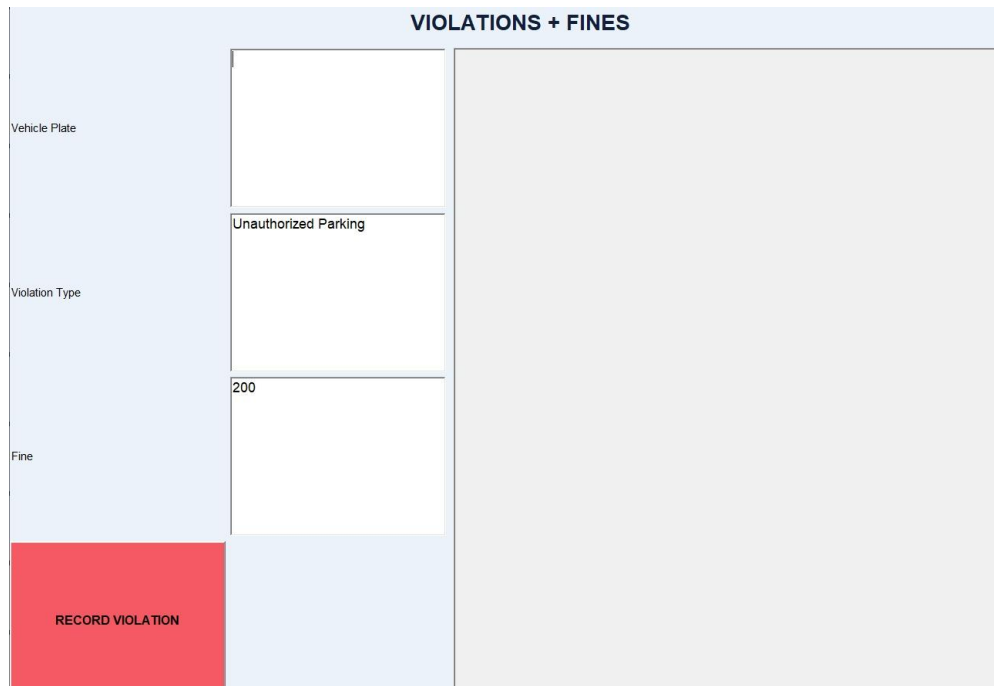
System.out.println("Violation recorded");
System.out.println("Fine Amount: " + fine);

```

```

package smartcampus;
import java.awt.*; import java.sql.*;
public class Violation extends Panel {
    TextField plate=UI.field("");
    type=UI.field("Unauthorized Parking"), fine=UI.field("200");
    TextArea out=new TextArea();
    Violation(Frame f){
        setLayout(new BorderLayout(8,8));
        setBackground(UI.BG);
        add(UI.Label("VIOLATIONS + FINES", 22, UI.NAVY),
            BorderLayout.NORTH);
        Panel p=new Panel(new GridLayout(0,2,5,5));
        p.add(new Label("Vehicle Plate"));
        p.add(plate);
        p.add(new Label("Violation Type"));
        p.add(type);
        p.add(new Label("Fine"));
        p.add(fine);
        Button b=UI.button("RECORD VIOLATION", UI.RED);
        p.add(b);
        add(p, BorderLayout.WEST);
        out.setEditable(false);
        add(out, BorderLayout.CENTER);
        b.addActionListener(e->save());
        void save(){
            try {
                Connection c=DB.get();
                PreparedStatement
                    p=c.prepareStatement("select vehicle_id from vehicles where plate_number=?");
                p.setString(1, plate.getText());
                ResultSet r=p.executeQuery();
                if(!r.next()){
                    out.setText("Vehicle not found");
                    return;
                }
                p=c.prepareStatement("insert into violations(vehicle_id, violation_type, fine_amount, status) values(?,?,?, 'Unpaid')");
                p.setInt(1, r.getInt(1));
                p.setString(2, type.getText());
                p.setDouble(3, Double.parseDouble(fine.getText()));
                p.executeUpdate();
                out.setText("Violation recorded successfully.");
            } catch (Exception e) {
                out.setText(e.getMessage());
            }
        }
    }
}

```



1.8 Reports and Analytics

```
BEGIN
READ selected report type
IF occupancy THEN calculate zone utilization
ELSE IF revenue THEN sum paid amounts
ELSE IF traffic THEN calculate grouped congestion summary
ELSE IF violations THEN group counts and fines
ELSE IF prediction baseline THEN calculate recent historical occupancy average
DISPLAY result
END
```

Algorithmic Logic Code

```
if (reportType.equals("occupancy")) {

String sql =
    "SELECT zone_id, COUNT(*) AS total_slots, " +
    "SUM(status='OCCUPIED') AS occupied " +
    "FROM parking_slots GROUP BY zone_id";

Statement st = con.createStatement();
ResultSet rs = st.executeQuery(sql);

while (rs.next()) {
    int zone = rs.getInt("zone_id");
    int total = rs.getInt("total_slots");
    int occupied = rs.getInt("occupied");

    double utilization = (occupied * 100.0) / total;

    System.out.println(
        "Zone: " + zone +
        " Utilization: " + utilization + "%"
```

```

    });
}

} else if (reportType.equals("revenue")) {

String sql =
    "SELECT SUM(amount) AS total_revenue " +
    "FROM payments WHERE payment_status='PAID'";

Statement st = con.createStatement();
ResultSet rs = st.executeQuery(sql);

```

```

package smartcampus;
import java.awt.*;import java.sql.*;
public class Reports extends Panel{
    TextArea out=new TextArea();
    Reports(Frame f){
        setLayout(new BorderLayout(8,8));setBackground(UI.BG);
        add(UI.Label("REPORTS & ANALYTICS",22,UI.NAVY),BorderLayout.NORTH);
        Panel p=new Panel(new GridLayout(0,1,5,5));
        Button a=UI.button("ZONE OCCUPANCY",UI.CYAN),
            b=UI.button("REVENUE",UI.GREEN),c=UI.button("TRAFFIC",UI.ORANGE),
            d=UI.button("VIOLATIONS",UI.RED),e=UI.button("PREDICTION",UI.PURPLE);
        p.add(a);p.add(b);p.add(c);p.add(d);p.add(e);
        add(p,BorderLayout.WEST);out.setEditable(false);add(out,BorderLayout.CENTER);
        a.addActionListener(x->zone());
        b.addActionListener(x->revenue());c.addActionListener(x->traffic());
        d.addActionListener(x->violations());e.addActionListener(x->prediction());
    }
    void zone(){run("select zone_name,total_slots,available_slots,occupied_slots,utilization_pct from v_zone_status",
        new String[]{"ZONE","TOTAL","FREE","OCC","UTIL%"});}
    void revenue(){try(Connection c=DB.get()){ResultSet r=c.createStatement().executeQuery("select coalesce(sum(amount),0) from payment
r.next();out.setText("TOTAL PAID REVENUE\nRs. "+r.getDouble(1));}catch(Exception e){out.setText(e.getMessage());}}
    void traffic(){run("select date(record_time),round(avg(congestion_level),1),count(*) from traffic_records group by date(record_time
        new String[]{"DATE","AVG CONGESTION","RECORDS"});}
    void violations(){run("select violation_type,count(*),coalesce(sum(fine_amount),0) from violations group by violation_type",new Str
    void prediction(){try(Connection c=DB.get()){ResultSet r=c.createStatement().executeQuery("select coalesce(round(avg(occupied_slots
    void run(String sql,String[] h){try(Connection c=DB.get()){ResultSet r=c.createStatement().executeQuery(sql);StringBuilder s=new St

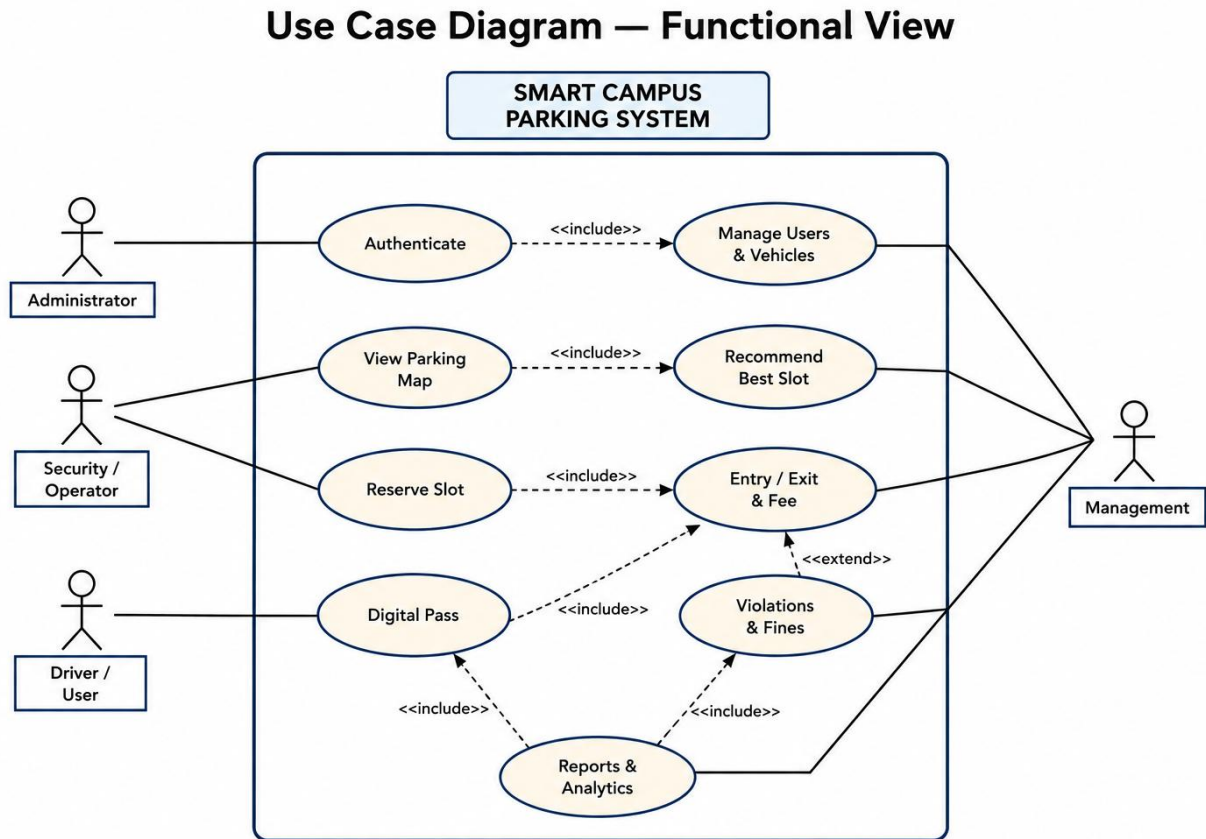
```

Output :

REPORTS & ANALYTICS	
ZONE OCCUPANCY	
REVENUE	
TRAFFIC	
VIOLATIONS	
PREDICTION	

UML / System Design Diagrams

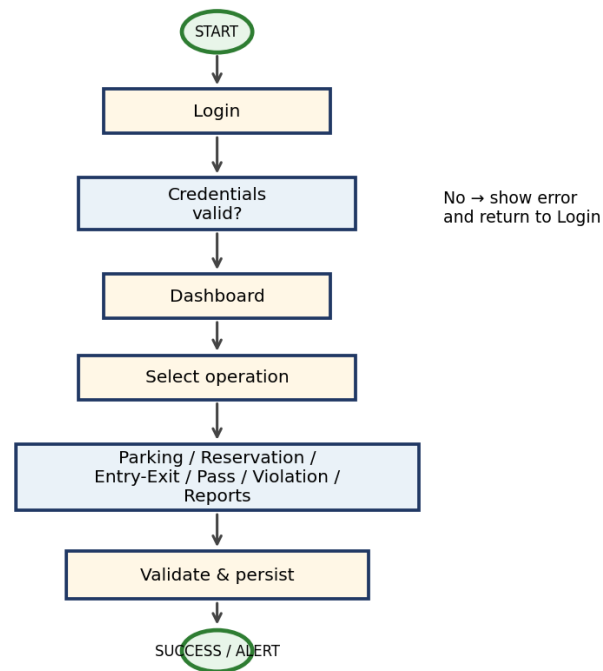
2.1 Use Case Diagram



The use case diagram identifies the main actors and system interactions: authentication, parking visualization, slot recommendation, reservation, vehicle movement, digital passes, violations and management reports.

2.2 Activity Diagram

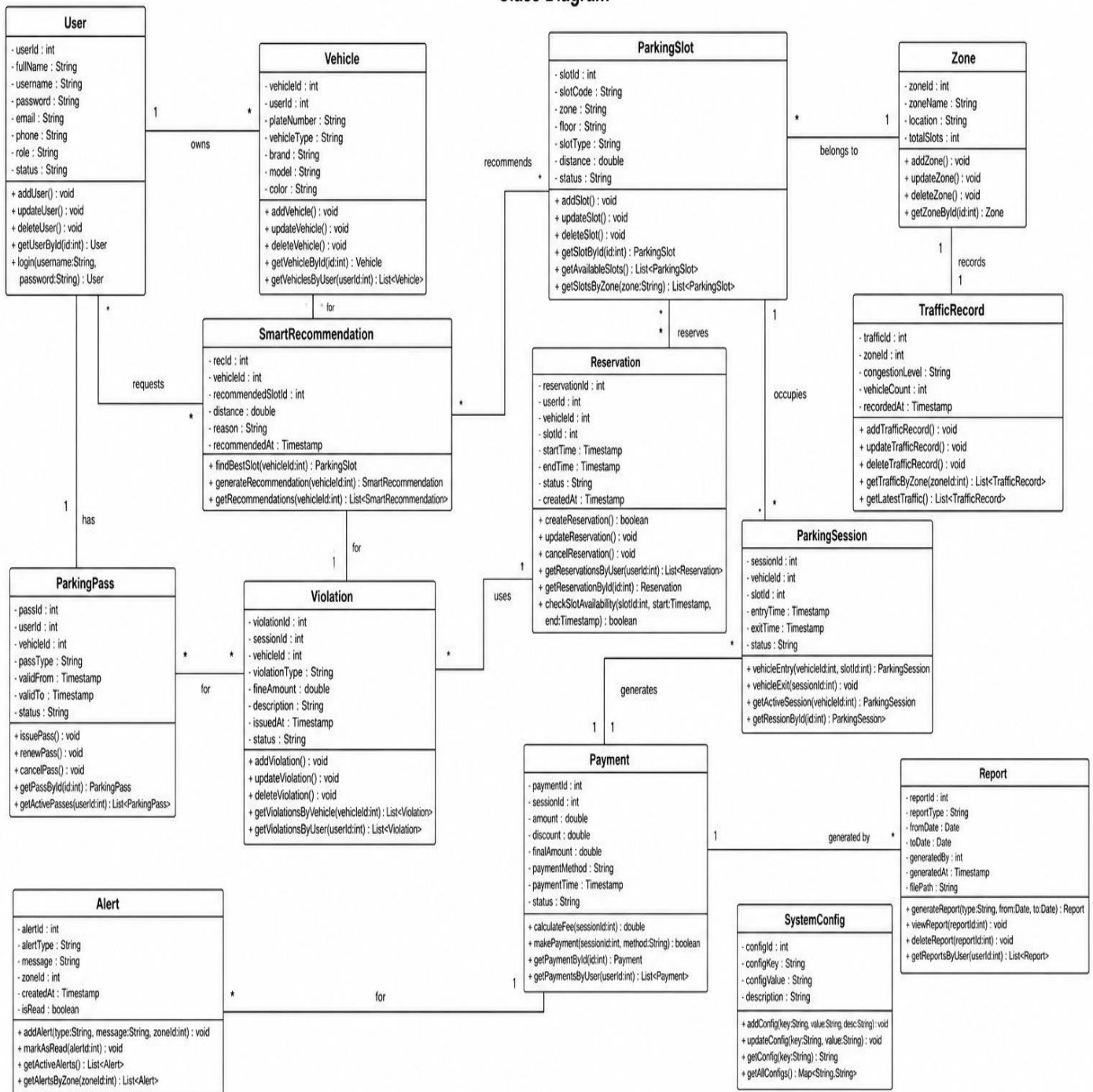
Activity Diagram — End-to-End Workflow



The activity diagram models the normal flow from login to dashboard, operation selection, validation, database persistence and success/alert output.

2.3 Class Diagram

Smart Campus Parking & Traffic Intelligence System
Class Diagram



The class diagram captures the core domain responsibilities and database-facing structure. UI/controller classes can be treated as additional implementation classes around these core concepts.

14. Individual Contribution Write-up

Jeyakannan L

192525376

During the development of the Smart Campus Parking & Traffic Intelligence System, my contribution focused on understanding the assigned parking-management problem and converting the requirements into a structured Java AWT and MySQL solution. I participated in planning the system modules and in maintaining the requirement that the graphical interface use AWT rather than Swing. I considered how the user interface, application logic and database should interact so that each operation produces a consistent result.

I contributed to the implementation of the AWT interface and event-driven operations. This involved working with components such as Frame, Panel, Button, Label, TextField, Choice, TextArea and graphical drawing facilities. I worked on the flow of user actions, input validation and the presentation of successful operations or error messages. I also contributed to parking functions such as slot availability, reservation handling, vehicle entry and exit, parking fee calculation, digital pass generation and violation recording.

E. Lakshmi Narayanan

192524249

I contributed to JDBC connectivity and SQL operations. I worked with database queries and parameterized operations and considered the appropriate use of Statement, PreparedStatement and CallableStatement. I also checked that transactions initiated from the AWT application were reflected in MySQL Workbench. This helped connect the frontend actions with persistent records such as reservations, parking sessions, payments and violations.

Testing and debugging were another part of my contribution. I checked normal and invalid inputs, reservation conflicts, unavailable parking slots, missing active sessions, database connectivity and report output. When errors occurred, I traced the event-handling logic and database operations to identify the source of the problem and verify the corrected behavior.

I also contributed to the documentation and presentation of the project by preparing pseudocode, system descriptions, UML diagrams, test cases and screenshot evidence. I supported the organization of the source files and database script for GitHub so that the project can be reproduced by another team member. Through this work, I gained practical experience in Java AWT GUI development, event handling, JDBC, MySQL, debugging, testing, documentation and collaborative software development.